

GitHub Copilot en VS Code y Proyecto Final

Integración de Herramientas e IA Autónoma

Autocompletado, Chat Integrado y Contexto (@workspace) en VS Code

¿Qué es el concepto?

A diferencia de la CLI, el editor VS Code me ofrece una integración profunda con GitHub Copilot mediante dos vías: el autocompletado en línea (texto fantasma) y el panel de Chat. El autocompletado me genera sugerencias basadas en mis archivos abiertos y el código adyacente sin consumir mis "AI Credits". Por otro lado, el Chat me permite usar modos como Ask (solo lectura) y Agent (lectura, ejecución y escritura). Además, usando directivas como @workspace, el agente puede indexar y consultar todo mi repositorio localmente para responderme preguntas arquitectónicas sin que tenga que pasarle los archivos uno por uno.

El proceso que realicé:

Configuré VS Code y validé la sesión de Copilot Pro. Probé el autocompletado en TaskService.java invocando un método de filtrado por prioridad; observé cómo Copilot infería la lógica basándose en el contexto circundante, y posteriormente descarté los cambios para mantener el entorno limpio. En el Chat, utilicé el modelo GPT-5 mini en modo Agent. Le instruí ejecutar la suite de pruebas mediante el prompt Corre mvn -q test en la terminal. Aprobé explícitamente el cuadro de diálogo de ejecución y verifiqué en la salida del chat el resultado exitoso de 77 pruebas, así como el consumo de créditos de la operación.

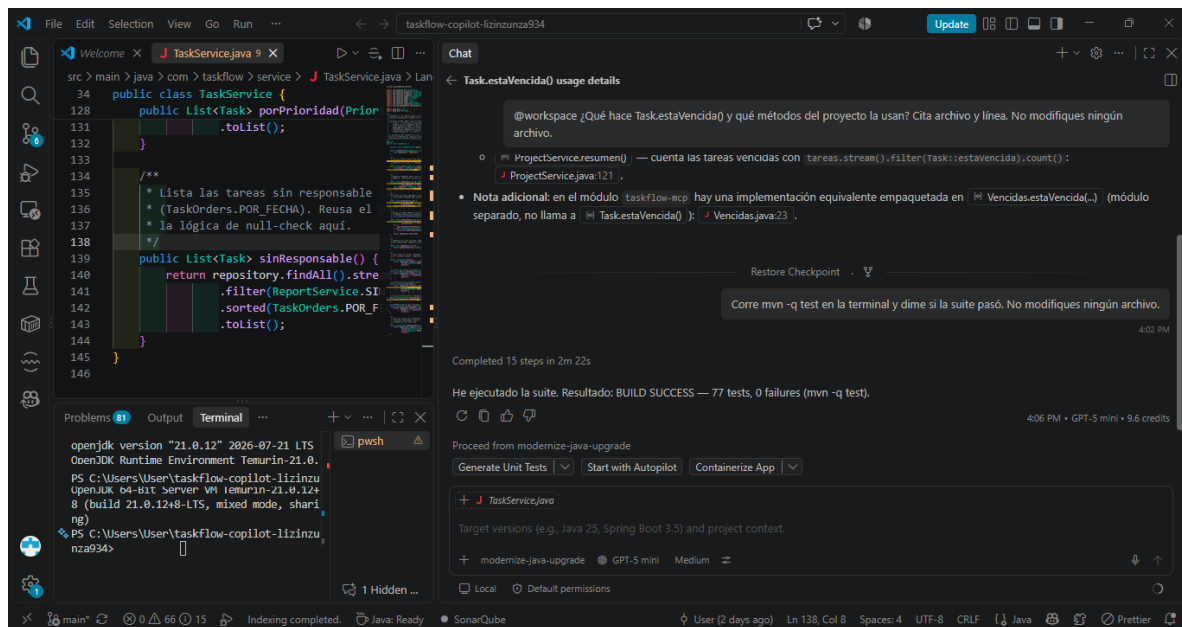


Ilustración 1 - Ejecución delegada de comandos de terminal mediante el Agente integrado en VS Code bajo aprobación explícita.

Migración de MCPs y Restricciones de Agentes Locales

¿Qué es el concepto?

VS Code gestiona los Model Context Protocol (MCP) de manera distinta a la CLI de Copilot. Mientras la CLI utiliza un archivo global en el perfil del usuario, VS Code lee un archivo local de entorno de desarrollo (.vscode/mcp.json). Los servidores locales se configuran bajo el transporte stdio y se ejecutan como procesos hijos ligados al ciclo de vida del editor. A su vez, los agentes definidos en .github/agents/ se integran nativamente en la interfaz gráfica, heredando las mismas restricciones de herramientas (Tools).

El proceso que realicé:

Migré la configuración MCP creando el archivo .vscode/mcp.json con los servidores taskflow, playwright y aws-knowledge. Inicié la aplicación Spring Boot en segundo plano e interactué con el servidor local a través del Chat invocando la herramienta listar_tareas_vencidas. Para comprobar la seguridad del entorno gráfico, seleccioné a mi agente revisor y le ordené modificar el código fuente. Tal como sucede en la CLI, el agente rechazó la orden de escritura indicando que carecía de permisos, lo cual validé ejecutando git status --porcelain.

Proyecto Final: Desarrollo y Verificación End-to-End

¿Qué es el concepto?

La sistematización del desarrollo asistido por IA requiere acoplar todos los elementos: especificaciones claras (Markdown), habilidades procedimentales (Skills), revisión por roles aislados (Agentes), validación real E2E (Scripts REST) y revisión de código automática (Copilot Code Review en GitHub). Este flujo garantiza que el código generado no solo compile y pase pruebas "falsas" (Mocks), sino que respete la lógica de negocio y las reglas arquitectónicas antes de integrarse a la rama principal.

El proceso que realicé:

Implementé la funcionalidad GET /tasks/search?q=api (Feature: search). Aislé el trabajo en una rama dedicada y utilicé el comando copilot -p acoplado a la skill crear-endpoint-taskflow. El modelo autogeneró el controlador, la lógica de servicio y 81 tests. Posteriormente, ejecuté al agente revisor, el cual detectó la necesidad de implementar una aserción verify(..., never()) y limpiar imports. Autoricé al agente a corregir sus propios hallazgos. Como validación final, integré mis casos de prueba al script verificar.ps1, logrando un resultado perfecto de 14/14 OK contra la aplicación real levantada. Empujé los cambios a GitHub, abrí un Pull Request y lo fusioné, documentando métricas y costos de IA en el documento final del proyecto (README.md).

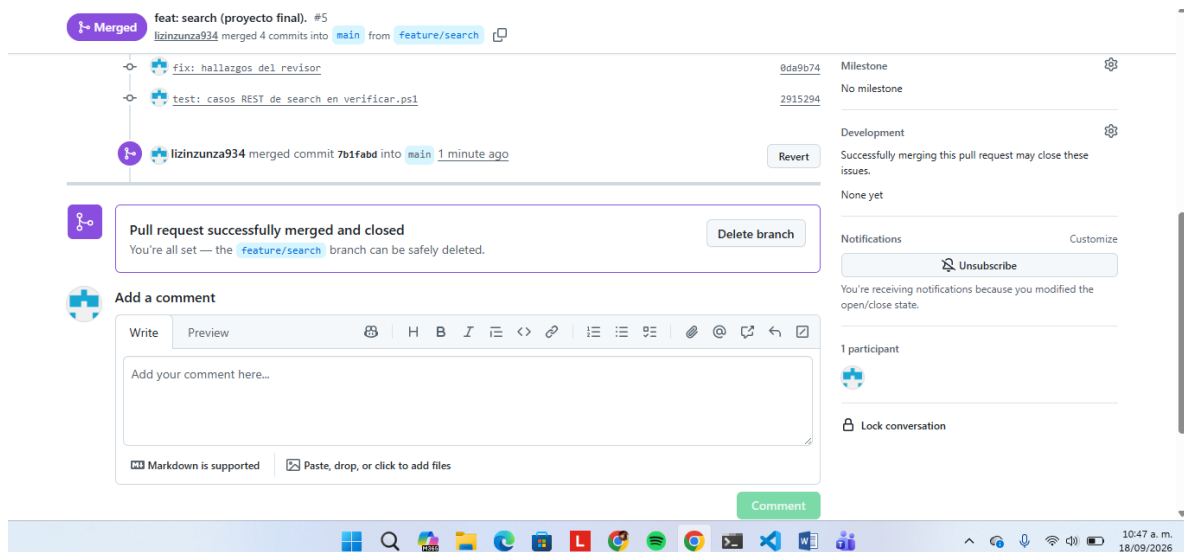


Ilustración 2 - Fusión del Pull Request en GitHub tras la validación funcional E2E de la feature desarrollada por IA.

Conclusión Final:

Las herramientas de IA como Copilot me sirven como multiplicadores de fuerza, pero no reemplazan mi trabajo en ingeniería de software. La diferencia en la sintaxis de MCP entre mi CLI y VS Code responde a las arquitecturas de aislamiento que manejo (global vs por proyecto). Esta semana comprobé que delegarle toda la responsabilidad al LLM es riesgoso por las inyecciones de prompt y las alucinaciones en los tests. Al final, lo que convierte a la IA en una herramienta segura y productiva es la estructura que le impongo: Skills que le guían el cómo, Agentes que le limitan el qué, y scripts de verificación que prueban la realidad.